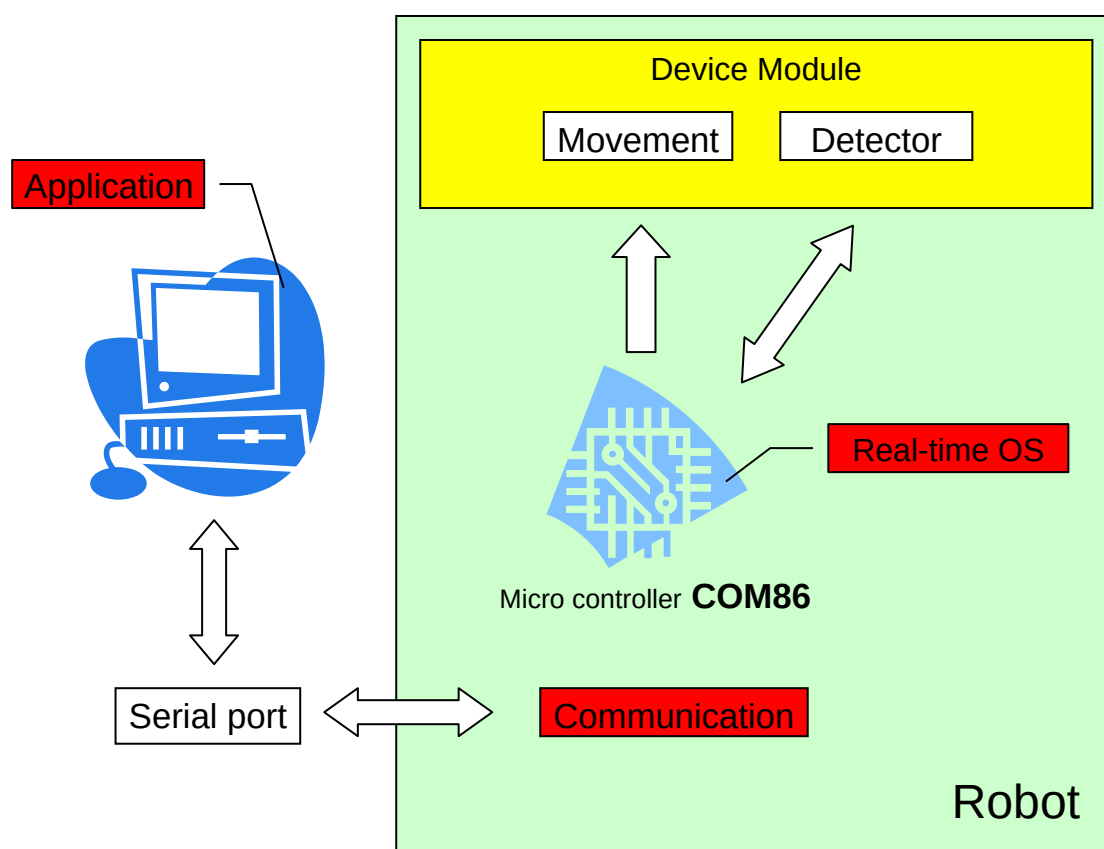


บทที่ 4

การออกแบบและพัฒนา

4.1 ขอบเขตของโครงการ

โครงการมีจุดประสงค์หลักคือ นำคอม86 มาประยุกต์ใช้ในงานหุ่นยนต์ และออกแบบโปรแกรมที่จะช่วยเหลือผู้ใช้งานในการสร้างหุ่นยนต์รวมถึงออกแบบชุดคำสั่งที่สามารถเรียกใช้ควบคุมอุปกรณ์ได้โดยง่าย รวมถึงโปรแกรมที่จะช่วยแสดงผลการทำงานและควบคุมหุ่นยนต์ผ่านทางคอมพิวเตอร์



รูปที่ 4-1 คอนเท็กซ์ไดอะแกรม ของโครงการ

จากภาพประกอบด้วยส่วนประกอบ 2 ส่วนด้วยกัน คือ

1. ส่วนของหุ่นยนต์ที่ใช้คอม86เป็นตัวประมวลผลหลักจากบอร์ดคอม86จะต่อเข้ากับ บอร์ดที่ทำการ ขยายพอร์ตแอดเดสและเชื่อมต่อกับ วงจรของอุปกรณ์ที่ออกแบบเป็นโมดูลมาตรฐานสำหรับเชื่อมต่อกับ ตัว บอร์ด ขยายพอร์ตแอดเดสและใช้ ระบบปฏิบัติการแบบเรียลไทม์ในการจัดการทำงานของคอม86โดยมีการสื่อสารระหว่างหุ่นยนต์กับคอมพิวเตอร์โดยการส่งข้อมูลผ่านพอร์ตอนุกรม
2. ส่วนของคอมพิวเตอร์ที่จะมีโปรแกรมหลักสำหรับควบคุมการทำงาน ส่วนคอมพิวเตอร์นั้นใช้สำหรับการเขียนโปรแกรมควบคุมหุ่นยนต์ด้วยภาษาซี ภายใต้การควบคุมโดยโปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์ ที่คณะผู้จัดทำสร้างขึ้นเพื่ออำนวยความสะดวกและจัดการการทำงานของหุ่นยนต์รวมถึงส่วนของชุดหับควบคุมและ

สั่งงานอุปกรณ์ ที่เป็นชุดคำสั่งสำหรับเรียกใช้งานเพื่อสะดวกและง่ายต่อการควบคุมอุปกรณ์บนตัวหุ่นยนต์ รวมถึงโปรแกรมที่ติดต่อส่งผ่านข้อมูลระหว่างหุ่นยนต์กับคอมพิวเตอร์ผ่านทางพอร์ตอนุกรม

จากส่วนประกอบทั้งสองด้านนั้นจำแนกตัวโครงงานออกเป็น 4 ส่วนหลัก ดังนี้

4.1.1 ส่วนของการเชื่อมต่ออุปกรณ์กับคอม86

โดยในส่วนนี้ทางคณะผู้จัดทำจะทำการออกแบบวงจรที่จะใช้งานตัวอุปกรณ์เช่น วงจรไคร์พมอเตอร์ วงจรเซนเซอร์ เป็นโมดูลที่มีรูปแบบมาตรฐาน ที่ทำให้ผู้ใช้งานสามารถเรียกใช้งาน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ได้อย่างถูกต้อง

4.1.2 ส่วนโปรแกรมควบคุมหลัก

โดยในส่วนนี้เป็นส่วนที่จะช่วยจัดการและดูแลการทำงานของหุ่นยนต์ประกอบด้วยส่วนต่าง ๆ ดังนี้

- ส่วนการจัดการอุปกรณ์ต่าง ๆ ที่นำมาเชื่อมต่อกับหุ่นยนต์
- ส่วนช่วยเหลือผู้ใช้งานในการเขียนโปรแกรมและโหลดโปรแกรมไปยังคอม86
- ส่วนควบคุมและจัดการเมมโมรีการใช้งานโปรแกรมต่าง ๆ บนคอม86
- ส่วนประมวลผลและส่งข้อมูลจากหุ่นยนต์ไปแสดงผลการทำงานที่คอมพิวเตอร์

4.1.3. ส่วนของคำสั่งสำหรับผู้ทำให้สามารถควบคุมอุปกรณ์ต่าง ๆ

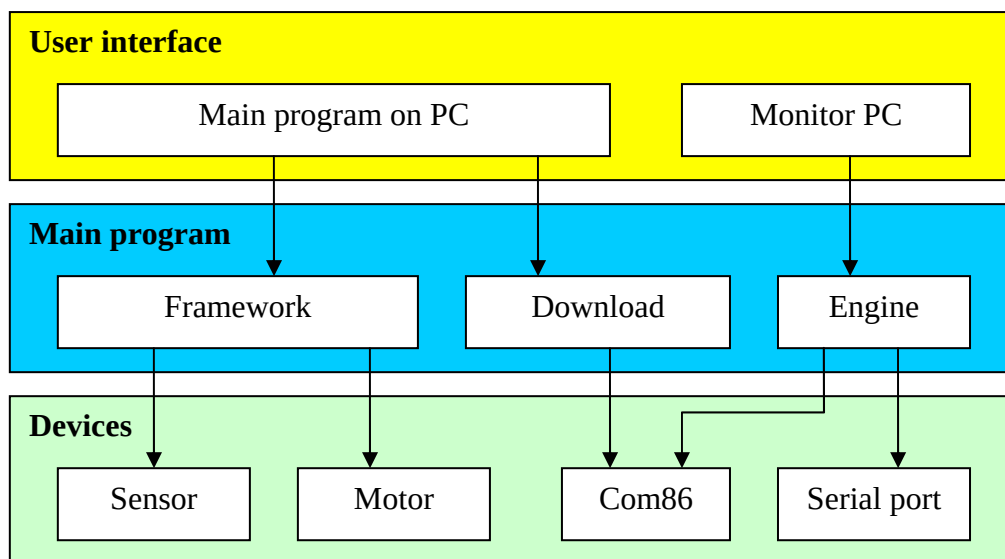
โดยในส่วนนี้ของ ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ นั้นทางคณะผู้จัดทำจะทำการเขียนขึ้น เป็นเหมือนชุดคำสั่งสำหรับตัวอุปกรณ์นั้นๆ โดยเมื่อผู้ใช้งานทำการเพิ่มอุปกรณ์นั้น ๆ เข้ามาทางโปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์ จะทำการโหลด .h ของอุปกรณ์ตัวนั้นมาลงในโปรแกรมหลัก (ตัวอย่างเช่น Include motor.h) และจะมีคำสั่งต่าง ๆ ที่ให้ผู้ใช้งานสามารถเรียกใช้ควบคุมอุปกรณ์ได้เลยโดยไม่จำเป็นต้องเขียนโปรแกรมติดต่อกับอุปกรณ์โดยตรง

4.1.4 ส่วนของโปรแกรมบนคอมพิวเตอร์

(ในขั้นตอนนี้จะทำการพัฒนาโปรแกรมที่ติดต่อกับหุ่นยนต์บนคอมพิวเตอร์ ก่อนแล้วอาจจะทำการเพิ่มเติมไปใช้งานบน Pocket PC หรือ PDA ในภายหลัง) โดยมีส่วนประกอบของโปรแกรมหลัก ๆ ดังนี้

- ส่วนหลักของโปรแกรม เป็นส่วน ติดต่อกับผู้ใช้งานหลักที่จะทำให้ผู้ใช้งาน สามารถเพิ่ม/ลดอุปกรณ์, เขียนโปรแกรม, ส่งข้อมูลไปยังคอม86 รวมถึงส่วนแสดงผลการทำงาน โดยคำสั่งต่าง ๆ ผู้ใช้งานสามารถใช้งานได้ตลอดเวลาเพราะบนหุ่นยนต์นั้นสามารถแก้ไขข้อมูลหรือเปลี่ยนแปลงโปรแกรมในขณะใดก็ได้
- ส่วนแสดงผลการทำงาน เป็นส่วนที่จะแสดงผลการทำงานของหุ่นยนต์ในขณะนั้นโดยจะมีโปรแกรมบนคอม86 จะทำการส่งข้อมูลต่าง ๆ มายังคอมพิวเตอร์ โดยบนคอมพิวเตอร์ จะมีโปรแกรมแสดงผลข้อมูลนั้น ๆ ให้กับผู้ใช้งานทำให้เราสามารถทราบถึงการทำงานและผลของการทำงานของหุ่นยนต์ตลอดเวลา

4.2 โครงสร้างทางสถาปัตยกรรมของโครงงาน



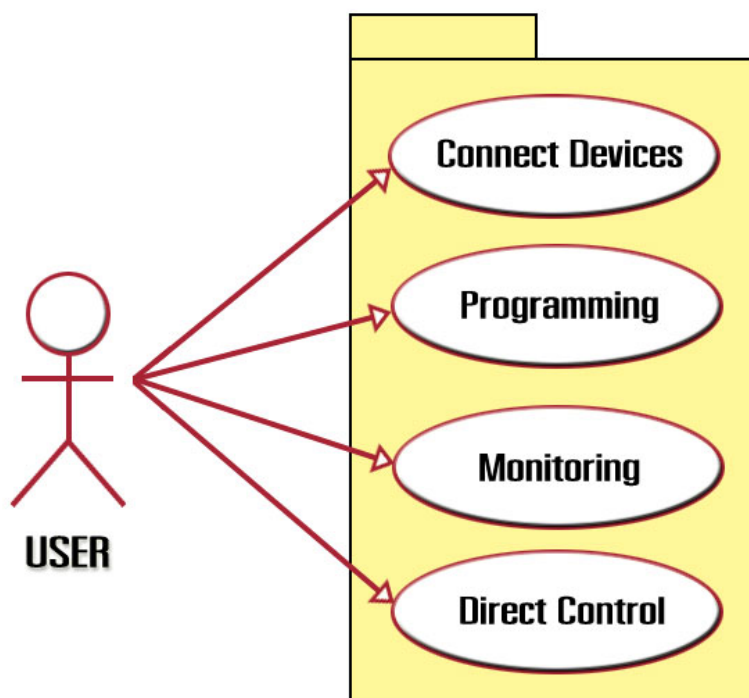
รูปที่ 4-2 โครงสร้างทางสถาปัตยกรรมของโครงการ

แบ่งโครงสร้างของโปรแกรมออกเป็น 3 ชั้น

- ส่วนแอปพลิเคชันบนคอมพิวเตอร์ที่ติดต่อกับผู้ใช้งาน (User Interface)
- ส่วนโปรแกรมควบคุมการทำงานหลัก (Main Program)
- ส่วนโปรแกรมที่ทำงานบนคอม86 ทำหน้าที่ควบคุมอุปกรณ์ (Devices layer)

โครงสร้างของโปรแกรมจำแนกลำดับชั้นของโครงสร้างโปรแกรมออกเป็น 3 ส่วนหลัก ดังภาพโดยส่วนที่ติดต่อกับผู้ใช้งานนั้นคือส่วนที่แอปพลิเคชันที่ทำงานอยู่บนคอมพิวเตอร์ โดยหน้าที่หลักของส่วนนี้คือช่วยในการสื่อสารระหว่างผู้ใช้งานกับหุ่นยนต์ อีกส่วนหลักก็คือ ส่วนโปรแกรมที่ทำงานติดต่อกับอุปกรณ์ซึ่งเป็นโปรแกรมที่ทำงานในระดับแอดเดสที่เชื่อมอยู่กับอุปกรณ์โดยส่วนนี้จะเหมือนส่วนชุดคำสั่งในระดับล่างที่ผู้ใช้งานสามารถเรียกใช้ได้โดยไม่ต้องทราบถึงการเชื่อมต่อในระดับฮาร์ดแวร์ ส่วนตรงกลางตามโครงสร้างที่วางไว้คือ เป็นส่วนของโปรแกรมควบคุมการทำงาน เป็นส่วนที่คอยจัดการและทำการช่วยเหลือผู้ใช้งาน ในส่วนของทำการเชื่อมต่อคอมพิวเตอร์ กับหุ่นยนต์นั่นเองโดยการทำงานส่วนใหญ่นั้นเป็น โปรแกรมที่ทำงานอยู่บนคอมพิวเตอร์

4.3 ยูสเคสไดอะแกรม (Use Case Diagram) ของระบบ



รูปที่ 4-3 ยูสเคสไดอะแกรมของผู้ใช้งาน

4.3.1 ผู้ใช้งาน (User)

ความต้องการจากผู้ใช้งานนับเป็นจุดสำคัญของโครงการเนื่องจากโปรแกรมมีจุดประสงค์หลัก มาจากการออกแบบโปรแกรมที่จะช่วยเหลือการสร้างหุ่นยนต์ ให้สามารถทำได้โดยง่ายสะดวก เข้าใจได้ง่ายและแก้ปัญหาการขาดทักษะรวมถึงขาดประสบการณ์ในการออกแบบ และทำงานกับอุปกรณ์ต่าง ๆ ของหุ่นยนต์

4.3.1.1 ติดตั้งอุปกรณ์กับบอร์ด คอม86

ผู้ใช้งานสามารถต่อโมดูลของอุปกรณ์ที่สนับสนุนการทำงานของฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ กับชุดพัฒนา คอม86 โดย ผู้ใช้งานจะต่อโมดูลของอุปกรณ์ กับบอร์ดขยายพอร์ตแอดเดส ที่ขยายพอร์ตจากชุดพัฒนา คอม86 ที่คณะผู้จัดทำกำหนดไว้ให้ และ ผู้ใช้งานจะสามารถปรับแต่ง บนแอปพลิเคชัน ด้วยว่าได้ต่ออุปกรณ์ใดไว้ที่ พอร์ต ไตบ้าง

4.3.1.2 เขียนโปรแกรมเพื่อใช้ควบคุมอุปกรณ์

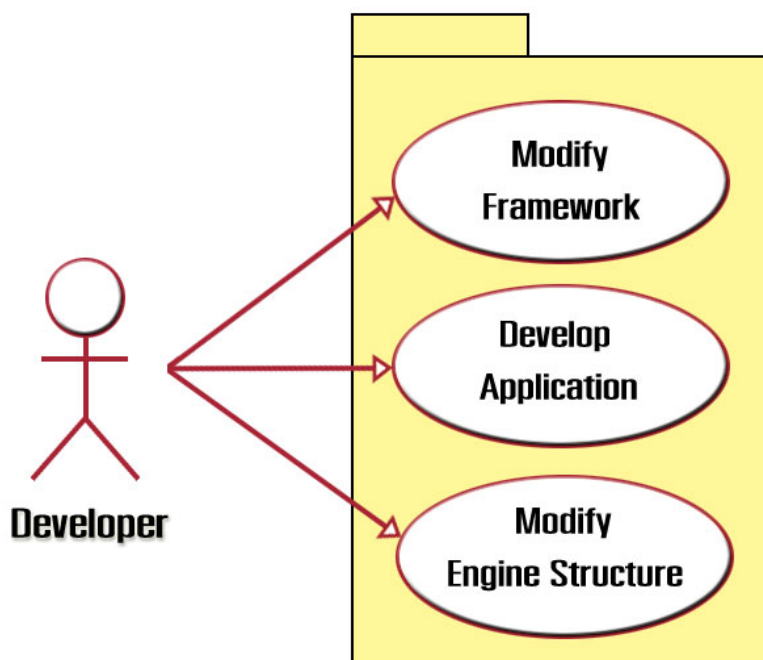
ผู้ใช้งาน สามารถเขียนโปรแกรม และดาวน์โหลดโปรแกรม ลงโดยใช้ฟังก์ชันจากชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ผ่าน แอปพลิเคชัน ที่คณะผู้จัดทำสร้างขึ้นมา เพื่อให้หุ่นยนต์สามารถทำงานตามโปรแกรม ใน การทำงานแบบอัตโนมัติได้

4.3.1.3 ดูสถานะของอุปกรณ์และการทำงานของหุ่นยนต์

ผู้ใช้งาน สามารถดูสถานะของอุปกรณ์ต่างๆ ที่มาเชื่อมต่อกับชุดพัฒนา คอม86 ได้ตลอดเวลาผ่านทางพอร์ตอนุกรม โดยจะทราบทันทีว่าอุปกรณ์แต่ละตัวมีการรับค่าหรือส่งค่าอะไรอยู่ เพื่อที่ ผู้ใช้งาน สามารถทราบถึงการทำงานของอุปกรณ์ และทำการตรวจสอบและแก้ไขโปรแกรม ให้เป็นไปอย่างถูกต้องได้

4.3.1.4 ควบคุมการทำงานของหุ่นยนต์โดยตรง

ผู้ใช้งาน สามารถควบคุมอุปกรณ์ที่เชื่อมต่อกับชุดพัฒนา คอม86 ได้โดยตรงในลักษณะของการทำงาน โดยตรง และผู้ใช้งานจะควบคุมอุปกรณ์ต่างๆผ่านทาง คีย์บอร์ด โดยตรง หรือ ผู้ใช้งาน ควบคุมอุปกรณ์โดยใช้ ฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ที่มีอยู่มาสั่งงานผ่านทางคอมพิวเตอร์โดยตรงได้



รูปที่ 4-4 ยูสเคสไดอะแกรมของผู้พัฒนา

4.3.2 ผู้นำไปพัฒนา (Developer)

4.3.2.1 แก้ไขและพัฒนา ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์

ผู้พัฒนา สามารถแก้ไขและเพิ่มเติมฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ที่คณะผู้จัดทำสร้างขึ้น เพื่อสามารถนำฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ไปใช้ตามความต้องการที่หลากหลายขึ้น และ เพื่อให้สามารถรองรับโมดูลของอุปกรณ์ชนิดใหม่ที่เพิ่มขึ้นมาได้ในอนาคต

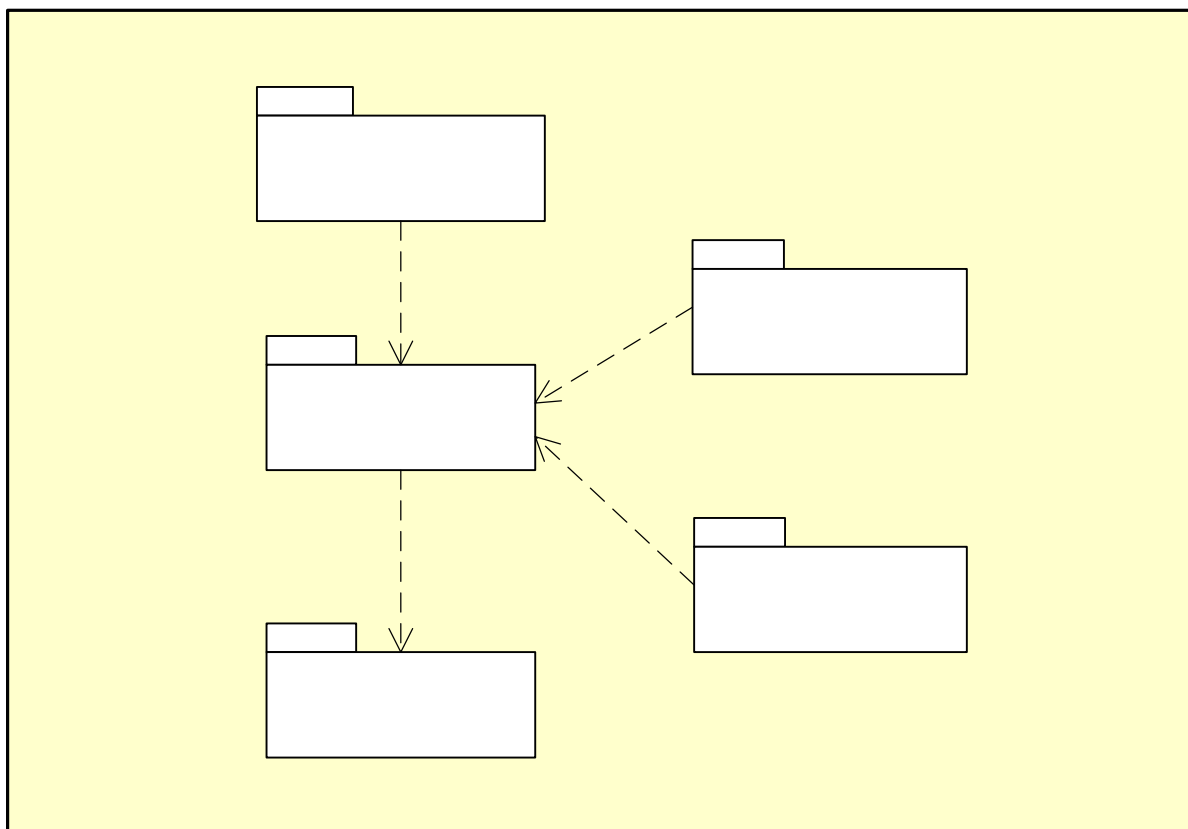
4.3.2.2 พัฒนาและเปลี่ยนแปลง GUI

ผู้พัฒนา สามารถแก้ไขและพัฒนา แอปพลิเคชัน ที่ใช้งานฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ที่คณะผู้จัดทำสร้างขึ้น ซึ่งอาจแก้ไขรูปแบบของ GUI หรือการทำให้เป็น แอปพลิเคชัน ใน pocket PC เพื่อควบคุม อุปกรณ์ที่เชื่อมต่อกับชุดพัฒนา คอม86 ได้

4.3.2.4 แก้ไขและปรับเปลี่ยนค่าของโปรแกรมควบคุมหลัก

ผู้พัฒนา สามารถแก้ไขโครงสร้างทาง Hardware บางอย่างของชุดพัฒนา คอม86ได้ เช่น เมมโมรี่ที่จองไว้ สำหรับ ผู้ใช้งาน โปรแกรม การแก้ไขตำแหน่งการเก็บค่าในรีจิสเตอร์ต่างๆ หรือ การปรับปรุงแก้ไขแอดเดส เพื่อ รองรับโมดูลของอุปกรณ์ชนิดใหม่ที่เพิ่มขึ้นมาได้

4.4 คอมโพเนนท์ไดอะแกรม (Component Diagram)



รูปที่ 4-5 แผนภาพคอมโพเนนต์ของโครงการ

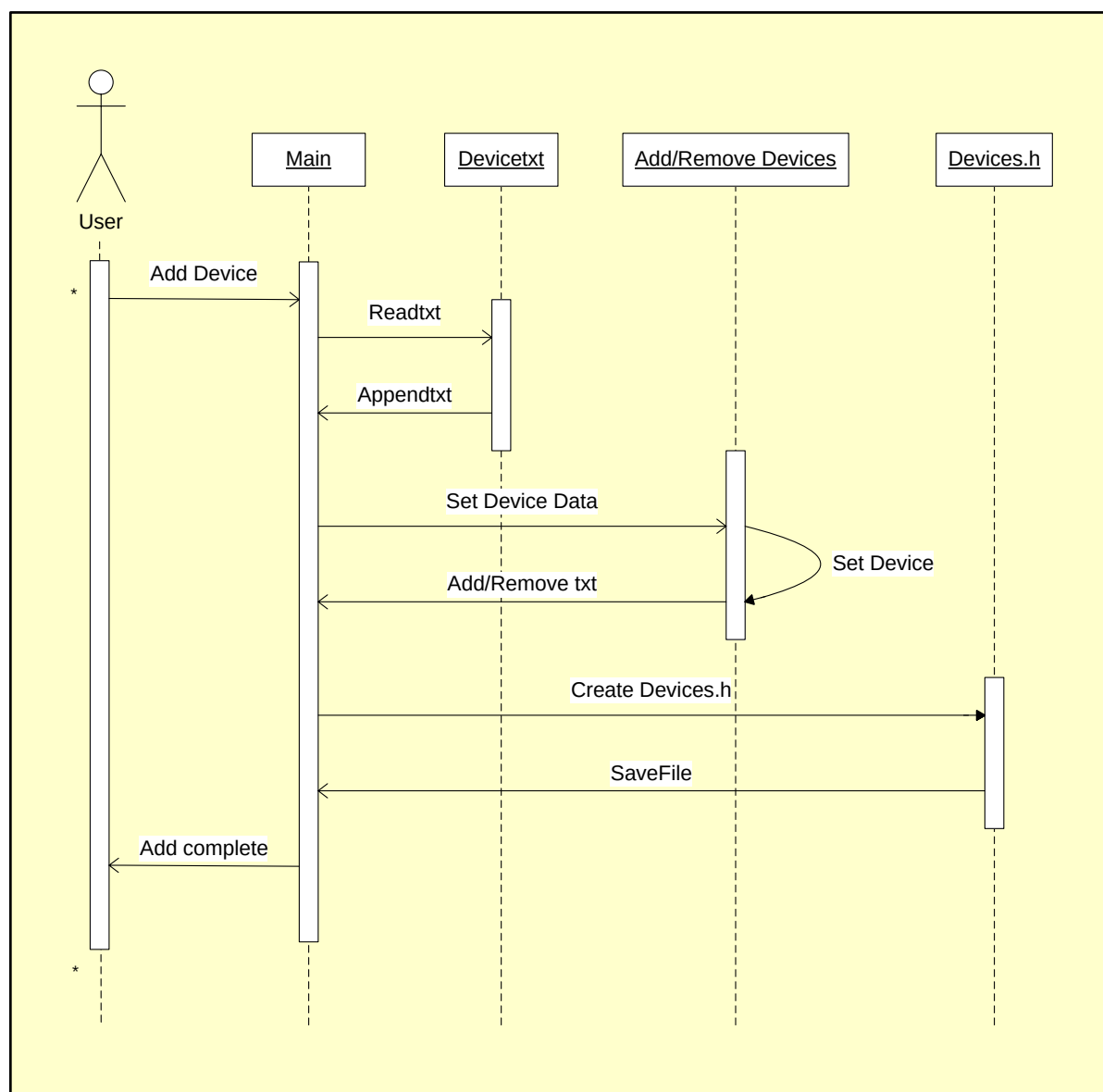
จากแผนภาพคอมโพเนนต์ที่กำหนดไว้จะทำการแบ่งหมวดหมู่ของโปรแกรมออกเป็น ส่วน ๆ โดยแต่ละส่วนจะมีการเรียกใช้อีกส่วนต่างกันไปตามลำดับ จากคอมโพเนนต์ใดอะแกรมที่ออกแบบขึ้นนั้น โปรแกรมหลักจะถูกแบ่งออกเป็น 5 ส่วนหลัก ดังนี้

- ส่วนของชุดคำสั่งที่เตรียมไว้สำหรับเรียกใช้ควบคุมอุปกรณ์ (Framework)
- ส่วนของโปรแกรมช่วยเหลือการทำงานโดยอัตโนมัติ (Engine)
- ส่วนของโปรแกรมที่ช่วยในการติดตั้งอุปกรณ์ (Connect Devices)
- ส่วนควบคุมหุ่นยนต์โดยตรง (Direct Control)
- ส่วนแสดงผลการทำงานจากหุ่นยนต์มายังคอมพิวเตอร์ (Monitoring)

4.5 จีเควนส์ไดอะแกรม (Sequence Diagram) ของระบบ

ในส่วนนี้จะทำการแสดงถึงขั้นตอนการทำงานของโปรแกรมแอปพลิเคชันบนคอมพิวเตอร์ ในส่วนต่าง ๆ ตามความต้องการ ที่ได้กำหนดในขั้นต้น โดยแผนภาพจะแสดงรายละเอียดขั้นตอนการทำงานของโปรแกรมควบคุมผ่านทางแอปพลิเคชันบนคอมพิวเตอร์รวมถึงขั้นตอนการเรียกใช้งานอุปกรณ์ต่าง ๆ ที่อยู่ในระบบ

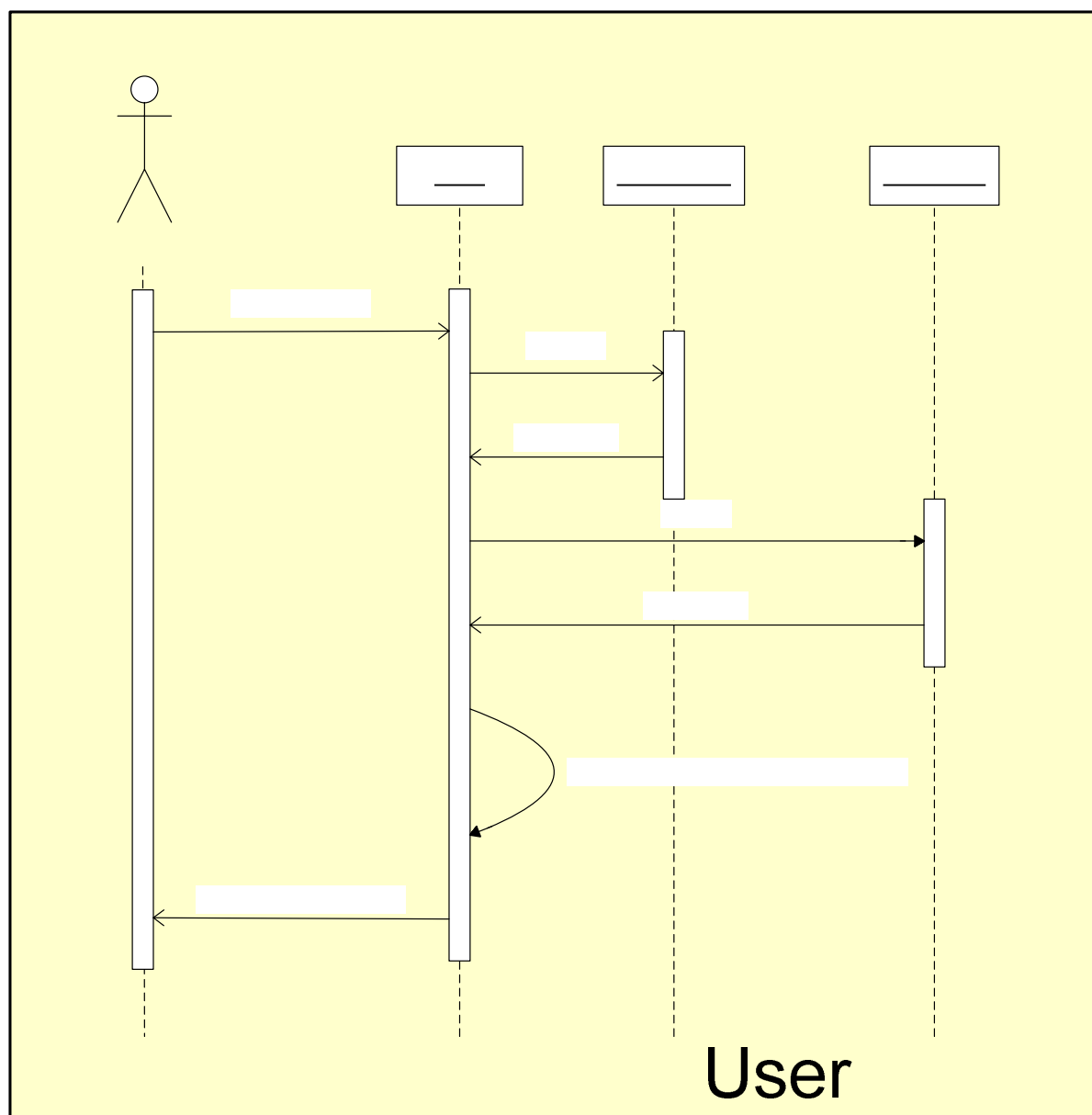
4.5.1 ติดตั้งอุปกรณ์กับบอร์ด คอม86 (Connect Devices)



รูปที่ 4-6 ซีเควนส์ไดอะแกรมของการติดตั้งอุปกรณ์ของผู้ใช้งาน

แผนภาพอธิบายการทำงานของโปรแกรมในส่วนการเชื่อมต่ออุปกรณ์กับคอม86 โดยผู้ใช้งานเริ่มจากเลือกที่จะติดตั้งอุปกรณ์โปรแกรมหลักจะทำการอ่านค่าไฟล์ข้อมูลอุปกรณ์ มาเก็บไว้เพื่อเตรียมทำการสร้างเฮดเดอร์ไฟล์ และหลังจากได้เลือกพอร์ตและอุปกรณ์จนครบแล้วโปรแกรมหลักจะทำการรวบรวมข้อมูลชนิดของอุปกรณ์พอร์ตที่ใช้แล้วทำการสร้างไฟล์ Devices.h ที่เป็นโลบารีของชุดคำสั่งที่เตรียมไว้สำหรับควบคุมอุปกรณ์ที่ติดตั้งไป

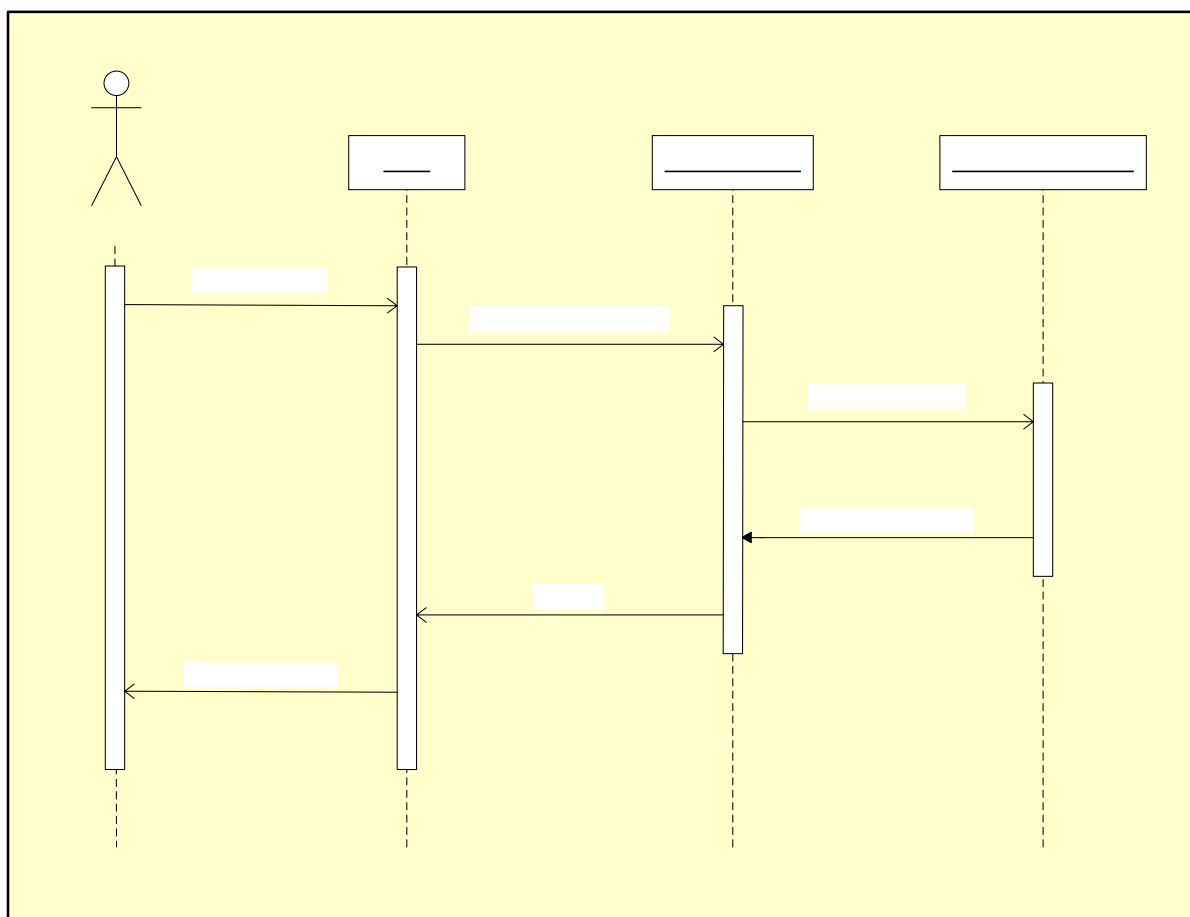
4.5.2 เขียนโปรแกรมเพื่อใช้ควบคุมอุปกรณ์ (Programming)



รูปที่ 4-7 ซีควเอนส์ไดอะแกรมของการเขียนโปรแกรมสั่งงานอุปกรณ์ของผู้ใช้งาน

แผนภาพอธิบายการทำงานของผู้ใช้งานเรียกใช้โปรแกรมหลักเพื่อทำการเขียนโปรแกรม โดยมีขั้นตอนคือ ผู้ใช้งานจะทำการเรียกแอปพลิเคชันว่าจะทำการเขียนโปรแกรม โดยโปรแกรมหลักจะเตรียมข้อมูลเพื่อทำการสร้างไฟล์ Program.cpp โดยการเรียกไฟล์ Devices.h แล้วทำการสร้างโครงสร้างโปรแกรมที่สนับสนุนการใช้งานระบบปฏิบัติการไมโครชิรุ่นที่ 2 พร้อมทั้งจะทำการสร้างออปเจกต์ของอุปกรณ์ตามข้อมูลที่ได้เก็บบันทึกไว้ จากนั้นโปรแกรมหลักก็จะทำการเพิ่มโปรแกรมในส่วนของการส่งค่าควบคุมหุ่นยนต์ และส่วนโปรแกรมแสดงผลการทำงานไว้ในส่วนท้ายของโปรแกรมแล้วทำการเซฟไฟล์เพื่อให้ผู้ใช้งานนำไฟล์ที่ได้ไปสร้างไฟล์ Program.exe ด้วยการคอมไพล์บน โปรแกรมมอร์แลนซี (Borland C)

4.5.3 ดูสถานะของอุปกรณ์และการทำงานของหุ่นยนต์ (Monitoring)



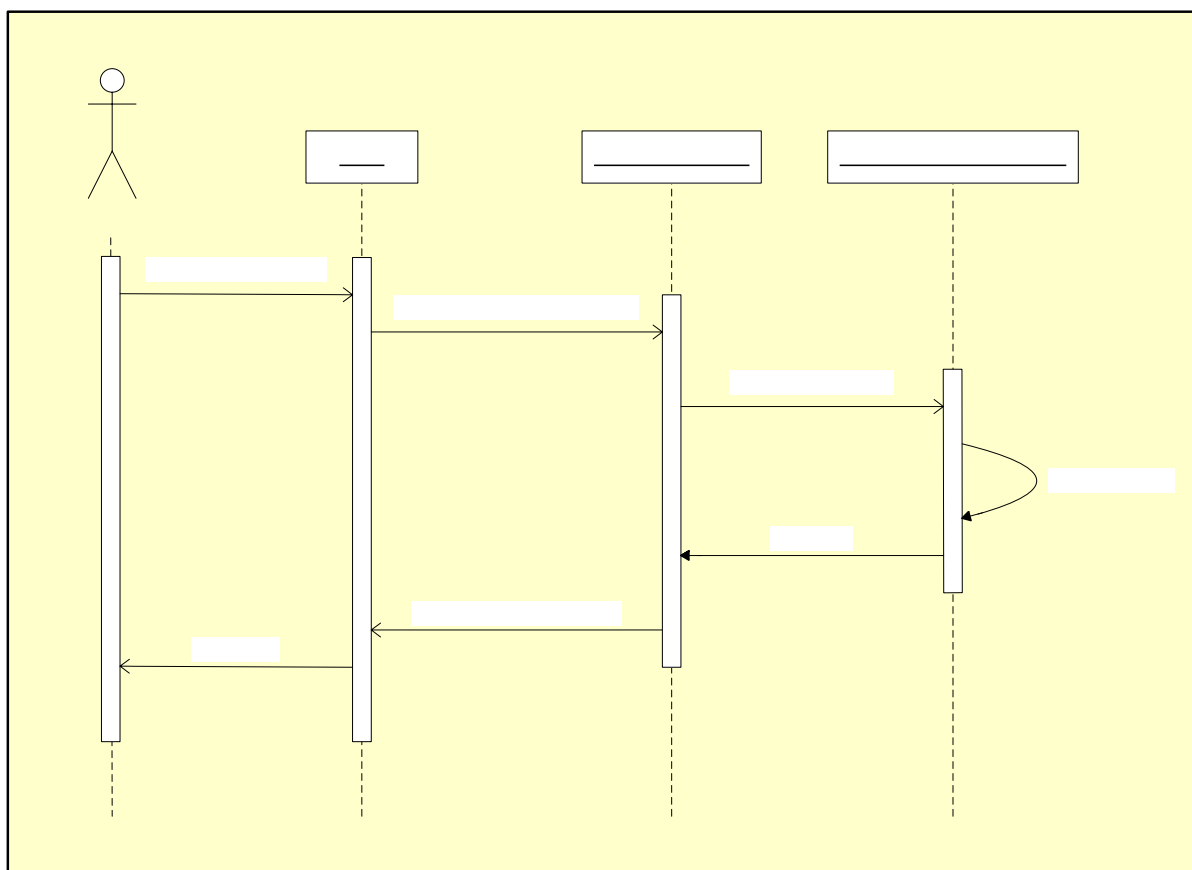
รูปที่ 4-8 ซีควেনส์ไดอะแกรมของการดูสถานะของอุปกรณ์และการทำงานของหุ่นยนต์

แผนภาพอธิบายการทำงานของผู้ใช้งานเรียกดูสถานะ การทำงานของอุปกรณ์ที่ติดตั้งอยู่บนหุ่นยนต์ โดยการเรียกดูค่าสถานะของหุ่นยนต์นั้นสามารถเรียกดูค่าผ่านทางแอปพลิเคชัน โปรแกรมบนคอมพิวเตอร์ได้ โดยการเรียกดูสถานะการทำงานของหุ่นยนต์นั้น เริ่มจากโปรแกรมหลักจะทำการเรียกใช้งานโปรแกรมแสดงการทำงานที่ทำงานอยู่บนคอม86 ผ่านทางพอร์ตอนุกรม โดยโปรแกรมแสดงผลที่ทำงานอยู่บนคอม86 จะทำการดึงข้อมูลสถานะของอุปกรณ์บนคอม86 ส่งไปยังคอมพิวเตอร์ผ่านทางพอร์ตเดียวกัน หลังจากโปรแกรมหลักทำการรับค่าจากคอม86 แล้วจะทำการตีความจากข้อมูลที่รับมาได้แล้วทำการแสดงผลการทำงานออกทางมอนิเตอร์ โดยรายละเอียดของการแสดงผลนั้นสามารถดัดแปลงได้ตามความต้องการ โดยโปรแกรมออกแบบมาช่วยเหลือในการช่วยรับค่าและส่งผ่านข้อมูล จากคอม86 มายังคอมพิวเตอร์เท่านั้น

User

Monitor Robot

4.5.4 ควบคุมการทำงานของหุ่นยนต์โดยตรง (Direct control)



รูปที่ 4-9 ซีควেনส์ไดอะแกรมของการควบคุมการทำงานของหุ่นยนต์โดยตรง

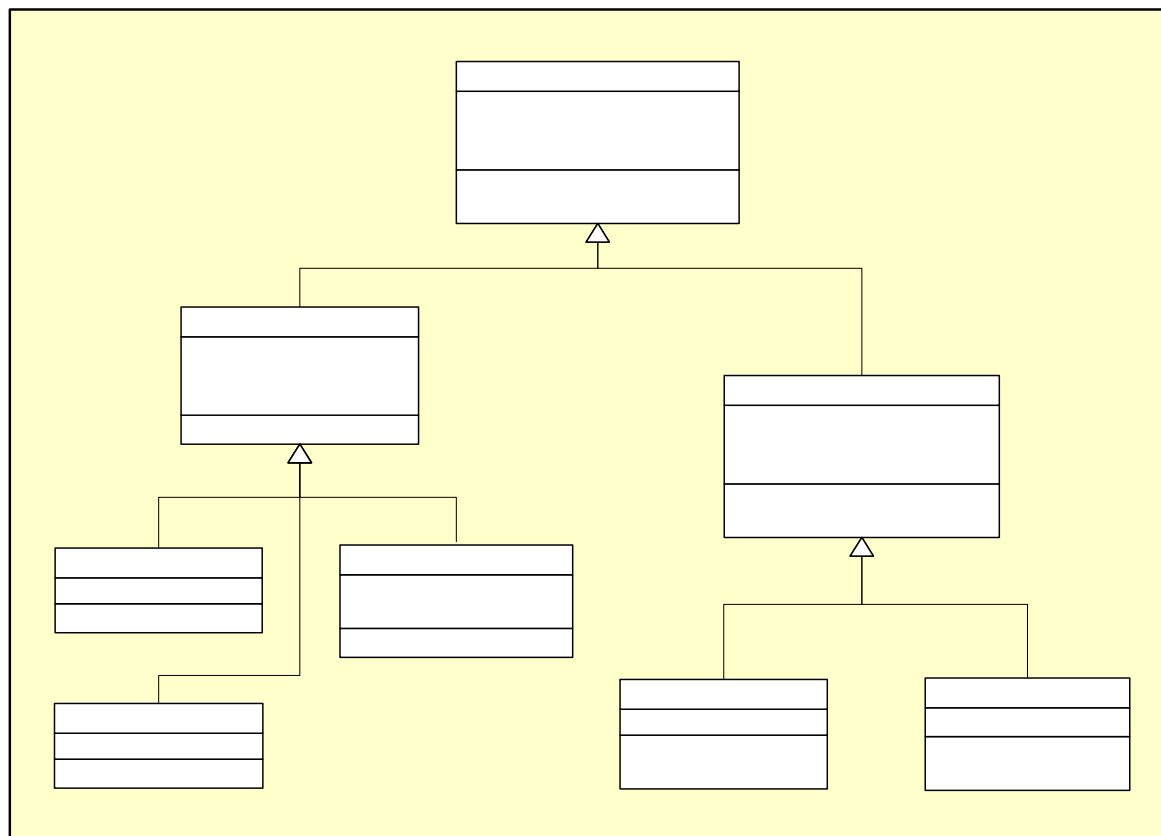
แผนภาพอธิบายการควบคุมหุ่นยนต์ โดยตรงจากคอมพิวเตอร์ในที่นี้ผู้ใช้งานสามารถส่งค่าไปยังคอม86 เพื่อทำการแก้ไขควบคุมหรือเปลี่ยนแปลงการทำงานของหุ่นยนต์ ได้ตลอดเวลา โดยรายละเอียดขั้นตอนการทำงานของโปรแกรมเริ่มจากผู้ใช้งานเรียกใช้โปรแกรมหลักโดยต้องการที่จะทำการควบคุมอุปกรณ์บนตัวหุ่นยนต์ จากนั้นโปรแกรมหลักจะทำการเรียกโปรแกรมควบคุมการทำงานของอุปกรณ์ที่ทำงานอยู่บนคอม86 โดยผ่านสายไปออก หลังจากคอม86ได้เปลี่ยนโหมดการทำงานแล้ว ผู้ใช้งานก็จะสามารถควบคุมหุ่นยนต์จากคอมพิวเตอร์โดยตรงผ่านทางพอร์ตอนุกรมได้

4.6 คลาสไดอะแกรม (Class Diagram) ของระบบ

คลาสไคอะแกรมของระบบจะแบ่งออกเป็น 2 ส่วนใหญ่ๆคือ

- คลาสไคอะแกรมของระบบในส่วนของคุณค่าสำหรับควบคุมอุปกรณ์
- คลาสไคอะแกรมของระบบในส่วนของโปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์

4.6.1 คลาสไคอะแกรมของระบบในส่วนของคุณค่าสำหรับควบคุมอุปกรณ์



รูปที่ 4-10 คลาสไคอะแกรมในส่วนของคุณค่าสำหรับควบคุมอุปกรณ์ (Framework)

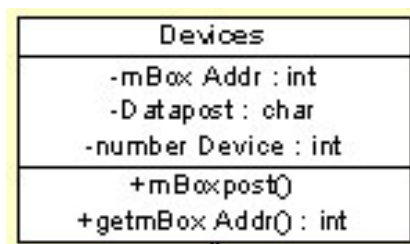
จากภาพเป็นการแสดงรายละเอียดของคลาสด้านในโปรแกรมที่เป็นส่วนของคุณค่าที่ใช้สำหรับควบคุมและใช้งานอุปกรณ์ ที่ผู้ใช้งานได้ติดตั้งไว้ ทางคณะผู้จัดทำได้ออกแบบชุดคำสั่งเป็น 2 ส่วนหลัก ด้วยกันคือส่วนของอุปกรณ์ที่ทำการรับค่าเข้ามายังตัวประมวลผล กับส่วนของอุปกรณ์ที่ทำการส่งข้อมูลไปสั่งการ จากภาพจะเห็นส่วนของมอเตอร์ และส่วนของเซนเซอร์แล้วรวมถึงรายละเอียดของคลาสที่ย่อยลงมาโดยในแต่ละคลาสที่ออกแบบนั้น มีรายละเอียดภายใน ดังต่อไปนี้

- คลาสมอเตอร์ ประกอบด้วยคลาสสเตปปีงมอเตอร์ และ ดีซีมอเตอร์
- คลาสเซนเซอร์ ประกอบด้วยคลาส อินฟราเรด , อัลตราโซนิก และสวิตช์แบบสัมผัส

โดยคลาสแม่ของโปรแกรมจะมีการเก็บค่าข้อมูลที่จำเป็นต่อการเรียกใช้งาน เช่น ชื่อของออปเจกของอุปกรณ์พอร์ตที่กำหนดทางลอจิกคอลโดยผู้ใช้งาน และค่าแอคเคสที่กำหนด สำหรับการส่งค่าไปยังเมลบล็อกของแต่ละทาสก์ของชุดคำสั่งที่จะทำการส่งค่าภายในแต่ละทาสก์ของระบบปฏิบัติการไมโครชิรุ่นที่2

ส่วนคลาสลูกที่ทำการอินเฮอริตแทนส์ (Inhabitant) มาจะประกอบด้วยฟังก์ชัน ที่เตรียมไว้สำหรับเรียกใช้เพื่อทำการควบคุมอุปกรณ์ โดยประกอบด้วยชุดคำสั่งสำหรับ ดีซีมอเตอร์, สเตปปีงมอเตอร์, อินฟราเรดเซนเซอร์, อัลตราโซนิก และสวิตช์สัมผัส ที่กำหนดเป็นอุปกรณ์พื้นฐาน

4.6.1.1 ลักษณะของคลาส Devices

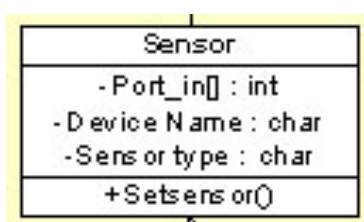


รูปที่ 4-11 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Devices

คลาส Devices เป็นคลาสพื้นฐานสำหรับอุปกรณ์ทุกชนิด โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ mBoxAddr เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าตำแหน่งของเมล์บ็อก ที่ใช้ในการส่งค่าข้อมูลควบคุมอุปกรณ์ระหว่างทาสก์ต่างในโปรแกรม
- แอตทริบิวต์ Datapost เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้เก็บค่าข้อมูลที่ส่งไปควบคุมอุปกรณ์แต่ละชนิด
- แอตทริบิวต์ number Device เป็นแอตทริบิวต์ชนิดตัวเลข ที่ใช้เก็บข้อมูลจำนวนอุปกรณ์แต่ละชนิดว่าอุปกรณ์ชนิดนั้นมีจำนวนเท่าไร
- เมธอด mBoxpost() เป็นเมธอดที่ใช้สำหรับส่งข้อมูลระหว่างกันในแต่ละทาสก์เพื่อนำข้อมูลเหล่านั้นไปใช้ควบคุมอุปกรณ์
- เมธอด getmBoxAddr() เป็นเมธอดที่ใช้สำหรับเรียกให้ส่งค่าแอตทริบิวต์ mBoxAddr กลับมา โดยมีการส่งค่ากลับมาเป็นตัวเลข

4.6.1.2 ลักษณะของคลาส Sensor



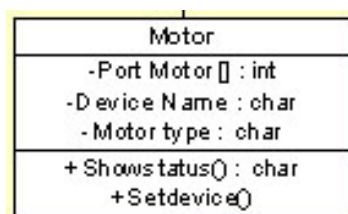
รูปที่ 4-12 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Sensor

คลาส Sensor เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิดที่เป็นอินพุตต่างๆ ที่สืบทอดมาจากคลาส Device โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ Port_in[] เป็นแอตทริบิวต์ชนิดชุดของตัวเลข ที่มีไว้สำหรับเก็บค่าตำแหน่งของอุปกรณ์โดยตำแหน่งของอุปกรณ์นี้จะใช้ในการอ้างถึงสำหรับควบคุม(รับข้อมูล)อุปกรณ์
- แอตทริบิวต์ DviceName เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้เก็บค่าชื่อที่ผู้ติดตั้งให้กับอุปกรณ์ต่างๆ
- แอตทริบิวต์ Sensortype เป็นแอตทริบิวต์ชนิดตัวอักษร ที่ใช้เก็บข้อมูลระบุชนิดของอุปกรณ์ เช่น เซอร์ว้า เป็นเซนเซอร์ชนิดใด

- เมทรูด Setsensor() เป็นเมทรูดที่ใช้สำหรับกำหนดค่าเริ่มต้นและกำหนดค่าแอตทริบิวต์ต่างๆของอุปกรณ์เซนเซอร์แต่ละชนิด

4.6.1.3 ลักษณะของคลาส Motor

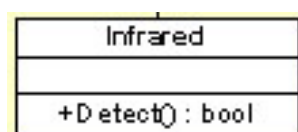


รูปที่ 4-13 รายละเอียดแอตทริบิวต์และเมทรูดของคลาส Motor

คลาส Motor เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิดที่เป็นเอาต์พุตต่างๆ ที่สืบทอดมาจากคลาส Device โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ PortMotor[] เป็นแอตทริบิวต์ชนิดชุดของตัวเลข ที่มีไว้สำหรับเก็บค่าตำแหน่งของอุปกรณ์โดยตำแหน่งของอุปกรณ์นี้จะใช้ในการอ้างถึงสำหรับควบคุม(ส่งข้อมูล)อุปกรณ์
- แอตทริบิวต์ DeviceName เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้เก็บค่าชื่อที่ใช้ตั้งให้กับอุปกรณ์ต่างๆ
- แอตทริบิวต์ Motortype เป็นแอตทริบิวต์ชนิดตัวอักษร ที่ใช้เก็บข้อมูลระบุชนิดของอุปกรณ์มอเตอร์ว่าเป็นมอเตอร์ชนิดใด
- เมทรูด Showstatus() เป็นเมทรูดที่ใช้ส่งค่าการทำงานปัจจุบันของมอเตอร์มาให้ผู้ใช้ทราบ
- เมทรูด Setdevice() เป็นเมทรูดที่ใช้สำหรับกำหนดค่าเริ่มต้นและกำหนดค่าแอตทริบิวต์ต่างๆของอุปกรณ์มอเตอร์แต่ละชนิด

4.6.1.4 ลักษณะของคลาส Infrared

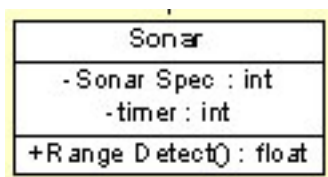


รูปที่ 4-14 รายละเอียดแอตทริบิวต์และเมทรูดของคลาส Infrared

คลาส Infrared เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิดอินฟราเรดเซนเซอร์ ที่สืบทอดมาจากคลาส Sensor โดยมีส่วนประกอบดังนี้

- เมทรูด Detect() เป็นเมทรูดที่ใช้ส่งค่าการทำงานปัจจุบันของอินฟราเรดเซนเซอร์มาให้ผู้นำไปใช้งาน โดยจะส่งค่ากลับมาเป็นค่าชนิดตรรกะ

4.6.1.5 ลักษณะของคลาส Sonar

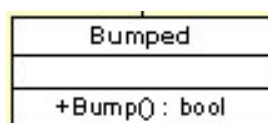


รูปที่ 4-15 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Sonar

คลาส Sonar เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิด อัลตราโซนิกเซนเซอร์ ที่สืบทอดมาจากคลาส Sensor โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ SonarSpec เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลการเปิด-ปิดการใช้งาน อัลตราโซนิกเซนเซอร์
- แอตทริบิวต์ timer เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้เก็บค่าเวลาที่ใช้นับหลังจากมีการส่งค่าข้อมูลของ อัลตราโซนิกเซนเซอร์ เพื่อนำใช้มาคำนวณระยะทาง
- เมธอด RangeDetect() เป็นเมธอดที่ใช้คำนวณและส่งค่าระยะทางที่ อัลตราโซนิกเซนเซอร์ วัดได้ไปให้ผู้ใช้งานไปใช้งาน โดยจะส่งค่ากลับมาเป็นค่าชนิดตัวเลขทศนิยม

4.6.1.6 ลักษณะของคลาส Bumped

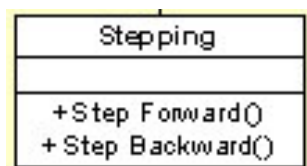


รูปที่ 4-16 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Bumped

คลาส Bumped เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิด บัมพ์เซนเซอร์ ที่สืบทอดมาจากคลาส Sensor โดยมีส่วนประกอบดังนี้

- เมธอด Bump() เป็นเมธอดที่ใช้ส่งค่าการทำงานปัจจุบันของ บัมพ์เซนเซอร์ มาให้ผู้ใช้งานไปใช้งาน โดยจะส่งค่ากลับมาเป็นค่าชนิดตรรกะ

4.6.1.7 ลักษณะของคลาส Stepping

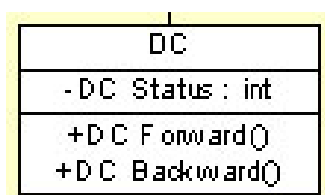


รูปที่ 4-17 รายละเอียดแอตทริบิวต์และเมทอดของคลาส Stepping

คลาส Stepping เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิด สเตปมอเตอร์ ที่สืบทอดมาจากคลาส Motor โดยมีส่วนประกอบดังนี้

- เมทอด StepForward() เป็นเมทอดที่ใช้ส่งคำสั่งงานให้ สเตปมอเตอร์ เคลื่อนที่ไปข้างหน้า
- เมทอด StepBackward() เป็นเมทอดที่ใช้ส่งคำสั่งงานให้ สเตปมอเตอร์ เคลื่อนที่กลับไปข้างหลัง

4.6.1.8 ลักษณะของคลาส DC

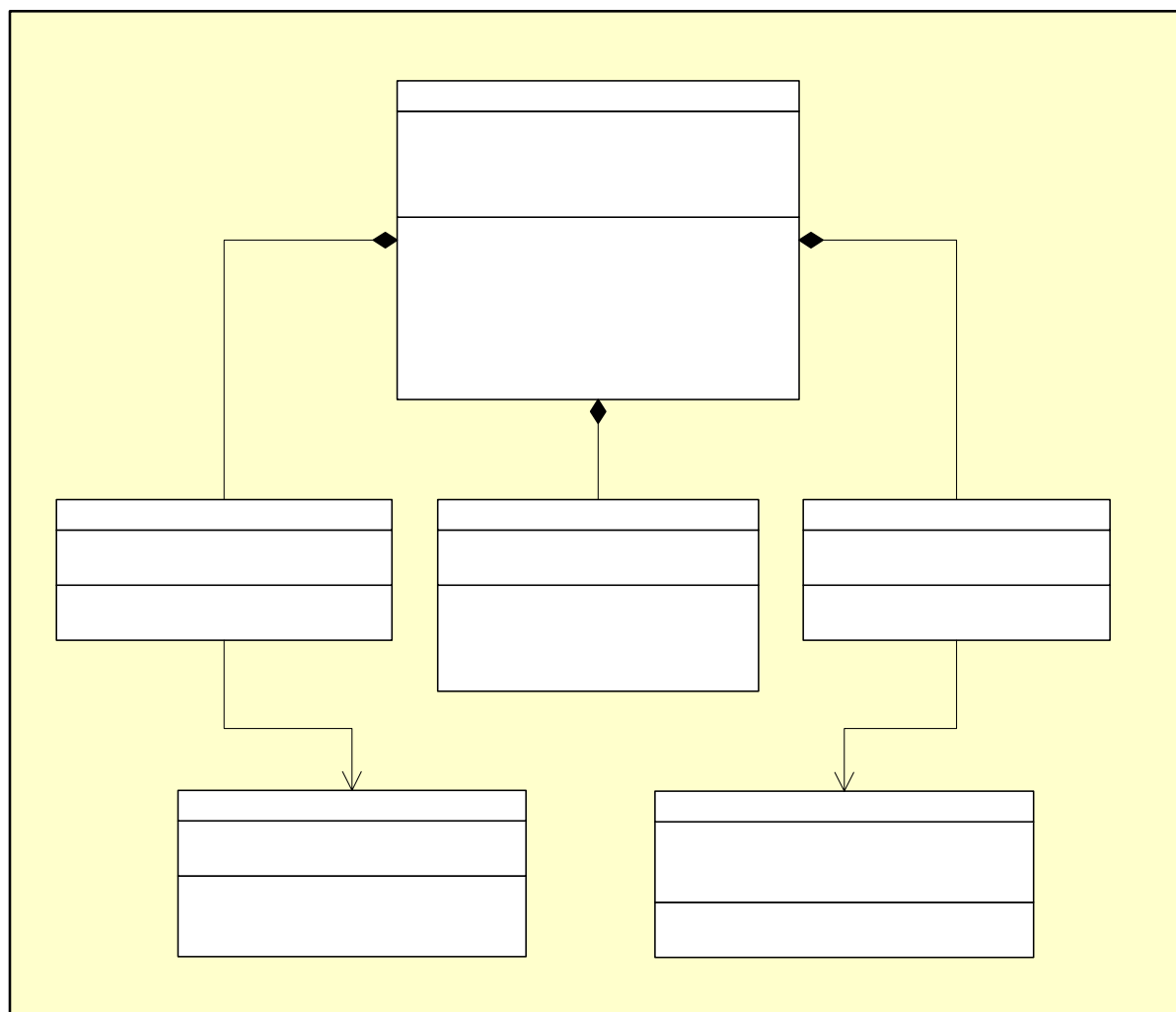


รูปที่ 4-18 รายละเอียดแอตทริบิวต์และเมทอดของคลาส DC

คลาส DC เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิด ดิซีมอเตอร์ ที่สืบทอดมาจากคลาส Motor โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ DCstatus เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะการทำงานของ ดิซีมอเตอร์
- เมทอด DCForward() เป็นเมทอดที่ใช้ส่งคำสั่งงานให้ ดิซีมอเตอร์ เคลื่อนที่ไปข้างหน้า
- เมทอด DCBackward() เป็นเมทอดที่ใช้ส่งคำสั่งงานให้ ดิซีมอเตอร์ เคลื่อนที่กลับไปข้างหลัง

4.6.2 คลาสไดอะแกรมของระบบในส่วนของโปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์



รูปที่ 4-19 คลาสไดอะแกรมในส่วน โปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์ (Engine)

จากภาพเป็นการแสดงรายละเอียดของคลาสที่เป็นส่วนประกอบภายในโปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์ ที่จะเป็นโปรแกรมที่จะทำการช่วยเหลือผู้ใช้งานให้สามารถจัดการเกี่ยวกับการติดตั้งอุปกรณ์ การเขียนโปรแกรม การสร้างโปรแกรมที่สามารถทำงานในแบบมัลติทาสก์ รวมถึงการเพิ่มโปรแกรมในส่วนการแสดงผลการทำงานและส่งค่าควบคุมการทำงานของหุ่นยนต์ผ่านทางแอปพลิเคชันบนคอมพิวเตอร์ โดยในแต่ละคลาสจะมีการทำงานดังต่อไปนี้

- คลาสของส่วนโปรแกรมหลัก
- คลาสของการติดตั้งอุปกรณ์
- คลาสของการรับค่าการทำงานมาแสดงผลบนคอมพิวเตอร์
- คลาสของการส่งข้อมูลควบคุมการทำงานของหุ่นยนต์โดยตรง

โดยในส่วนของคลาสแสดงผลการทำงาน และคลาสควบคุมการทำงานนั้น จะแบ่งออกเป็นคลาสย่อยที่ทำงานอยู่บนคอม86 ด้วยอีกส่วนหนึ่งเพื่อให้โปรแกรมหลักนั้น สามารถเรียกใช้ให้หุ่นยนต์สามารถส่งข้อมูลและรับข้อมูลมายังคอมพิวเตอร์ผ่านทางพอร์ตอนุกรม

4.6.2.1 ลักษณะของคลาส MainEngine

Main Engine
-Devicetxt : char -txt Name : char -Enginetxt : char -Device Name : char
+Readtxt() : String +Writetxt() : String +Create Device.h() : String +Create Direct Program () : void +Create Monitor Program () : void +Savefile() : String +Openfile() : String

รูปที่ 4-20 รายละเอียดแอตทริบิวต์และเมทอดของคลาส MainEngine

คลาส MainEngine เป็นคลาสหลักของส่วนโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ Devicetxt เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของแฟ้มข้อมูลของอุปกรณ์ต่างๆ
- แอตทริบิวต์ txtName เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของแฟ้มข้อมูลโปรแกรมหลักที่สร้างจากโปรแกรมช่วยเหลือผู้ใช้
- แอตทริบิวต์ Enginetxt เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของแฟ้มข้อมูลของส่วนประกอบของโปรแกรมช่วยเหลือผู้ใช้
- แอตทริบิวต์ DeviceName เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของอุปกรณ์ต่างๆ
- เมทอด Readtxt() เป็นเมทอดที่ใช้อ่านแฟ้มข้อมูลเพื่อนำมาประมวลผลสร้างโปรแกรมหลัก
- เมทอด Writetxt() เป็นเมทอดที่ใช้เขียนค่าลงไปในแฟ้มข้อมูลโปรแกรมหลัก
- เมทอด CreateDevice.h() เป็นเมทอดที่ใช้สร้างแฟ้มข้อมูลแบบเฮดเดอร์ ของอุปกรณ์ที่ผู้ใช้เลือก
- เมทอด CreateDirectProgram() เป็นเมทอดที่ใช้สร้างส่วนประกอบแฟ้มข้อมูลโปรแกรมหลักในส่วนทาสก์ที่ใช้ควบคุมอุปกรณ์โดยตรงจากผู้ใช้
- เมทอด CreateMonitorProgram() เป็นเมทอดที่ใช้สร้างส่วนประกอบแฟ้มข้อมูลโปรแกรมหลักในส่วนทาสก์ที่ใช้แสดงผลสถานะการทำงานของอุปกรณ์ต่างๆ
- เมทอด Savefile() เป็นเมทอดที่ใช้บันทึกแฟ้มข้อมูลโปรแกรมหลักที่โปรแกรมช่วยเหลือสร้างขึ้นมาแล้วผู้ใช้นำไปเขียนโปรแกรมในส่วนของทาสก์ผู้ใช้
- เมทอด Openfile() เป็นเมทอดที่ใช้เปิดแฟ้มข้อมูลโปรแกรมหลักที่ผู้ใช้เคยบันทึกไว้แล้ว

4.6.2.2 ลักษณะของคลาส DirectControl

Direct Control
-Key input : char -Mode control : int
+getKey() : char +Create control Data() : char

รูปที่ 4-21 รายละเอียดแอตทริบิวต์และเมธอดของคลาส DirectControl

คลาส DirectControl เป็นคลาสที่เป็นส่วนประกอบของคลาส MainEngine โดยจะทำงานในส่วนการควบคุมอุปกรณ์โดยตรงของโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ Keyinput เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าอินพุตที่รับจากผู้ใช้งานโดยวิธีการกดคีย์บอร์ด
- แอตทริบิวต์ Modecontrol เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะการทำงานของการทำงานของอุปกรณ์
- เมธอด getKey() เป็นเมธอดที่ใช้รับค่าข้อมูลจากผู้ใช้งานผ่านทางคีย์บอร์ด
- เมธอด CreatecontrolData() เป็นเมธอดที่ใช้สร้างข้อมูลที่ส่งไปควบคุมอุปกรณ์แต่ละชนิดโดยตรง

4.6.2.3 ลักษณะของคลาส Add/Remove Device

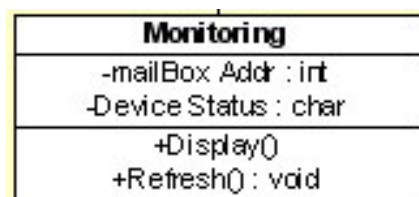
Add/Remove Device
-Device Port : int -Device mailBox : int
+set Device() : void +initial Device() : char +set Port() : void +set mailBox() : void

รูปที่ 4-22 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Add/Remove Device

คลาส Add/Remove Device เป็นคลาสที่เป็นส่วนประกอบของคลาส MainEngine โดยจะทำงานในส่วนการติดตั้งอุปกรณ์ของโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ DevicePort เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลตำแหน่งที่ใช้อ้างอิงถึงอุปกรณ์ต่างๆ ที่ผู้ใช้เลือกไว้
- แอตทริบิวต์ DevicemailBox เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลแม่ล้น็อกที่ใช้ส่งข้อมูลของแต่ละอุปกรณ์
- เมธอด setDevice() เป็นเมธอดที่ใช้กำหนดชนิดอุปกรณ์ที่ผู้ใช้เลือก
- เมธอด initialDevice() เป็นเมธอดที่ใช้กำหนดค่าเริ่มต้นของอุปกรณ์ที่ผู้ใช้เลือก
- เมธอด setPort() เป็นเมธอดที่ใช้กำหนดค่าตำแหน่งที่ใช้อ้างอิงถึงอุปกรณ์ที่ผู้ใช้เลือก
- เมธอด setmailBox() เป็นเมธอดที่ใช้กำหนดค่าแม่ล้น็อกให้กับอุปกรณ์ทุกตัวที่ผู้ใช้เลือก

4.6.2.4 ลักษณะของคลาส **Monitoring**

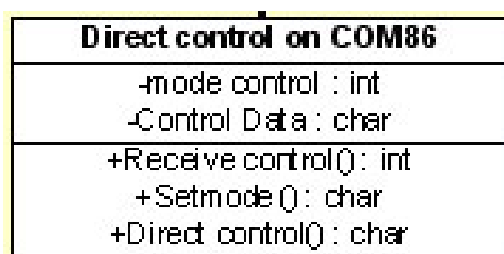


รูปที่ 4-23 รายละเอียดแอตทริบิวต์และเมธอดของคลาส *Monitoring*

คลาส *Monitoring* เป็นคลาสที่เป็นส่วนประกอบของคลาส *MainEngine* โดยจะทำงานในส่วนการแสดงผลสถานะของอุปกรณ์ต่างให้ผู้ใช้ทราบทางจอแสดงผลของโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ *mailBoxAddr* เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลตำแหน่งเมล์บ็อกที่ใช้ส่งข้อมูลของแต่ละอุปกรณ์
- แอตทริบิวต์ *DeviceStatus* เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะการทำงานของอุปกรณ์แต่ละตัว
- เมธอด *Display()* เป็นเมธอดที่ใช้แสดงผลสถานะของอุปกรณ์ต่างทางจอแสดงผล
- เมธอด *Refresh()* เป็นเมธอดที่ใช้เปลี่ยนค่าที่แสดงผลใหม่

4.6.2.5 ลักษณะของคลาส **Direct control on Com86**



รูปที่ 4-24 รายละเอียดแอตทริบิวต์และเมธอดของคลาส *Direct control on Com86*

คลาส *Direct control on Com86* เป็นคลาสย่อยที่เกี่ยวข้องกับคลาส *DirectControl* โดยจะทำงานในส่วนการควบคุมอุปกรณ์โดยตรงที่จะทำงานภายในคอม86 ของโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ *modecontrol* เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะการควบคุมอุปกรณ์ของโปรแกรมช่วยเหลือผู้ใช้งาน
- แอตทริบิวต์ *ControlData* เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าข้อมูลที่ส่งไปสั่งงานอุปกรณ์ต่างๆโดยตรง
- เมธอด *Receivecontrol()* เป็นเมธอดที่รับค่าข้อมูลคำสั่งควบคุมอุปกรณ์จากโปรแกรมช่วยเหลือผู้ใช้งานที่อยู่บนคอมพิวเตอร์
- เมธอด *Setmode()* เป็นเมธอดที่รับค่าข้อมูลการกำหนดสถานะการควบคุมโปรแกรมช่วยเหลือผู้ใช้งานที่อยู่บนคอมพิวเตอร์มากำหนดให้กับโปรแกรมที่กำลังทำงานอยู่ที่คอม86

- เมธอด Directcontrol() เป็นเมธอดที่ใช้ส่งค่าข้อมูลคำสั่งควบคุมอุปกรณ์จากคอม86ไปยังตัวอุปกรณ์นั้นๆ

4.6.2.6 ลักษณะของคลาส **Monitoring on Com86**

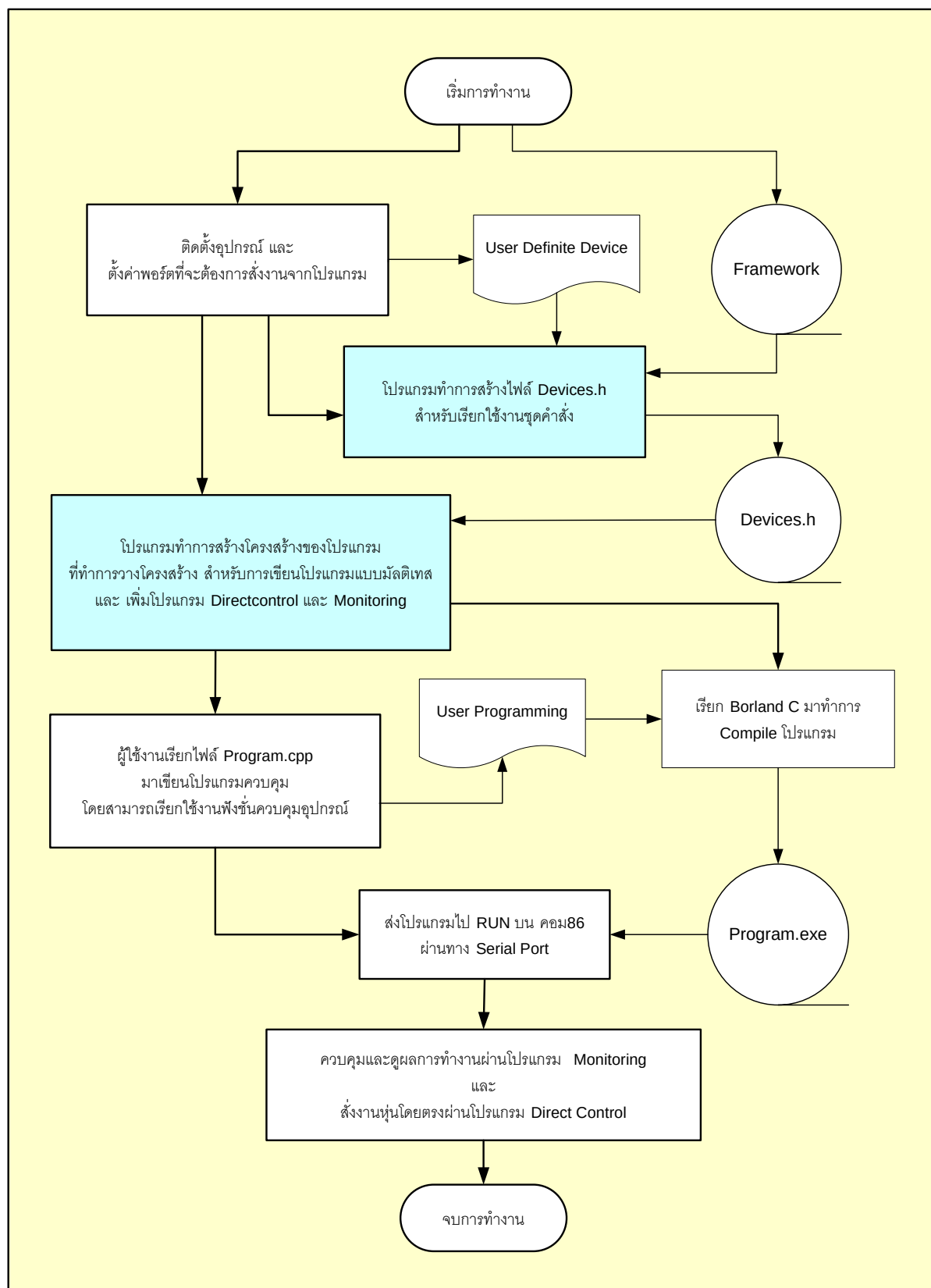
Monitoring on COM86
-mailBox Addr : int
-Device Name : char
-Monitor Data : char
+SendMonitor Data() : char
+GetMonitor Data() : char

รูปที่ 4-25 รายละเอียดแอตทริบิวต์และเมธอดของคลาส *Monitoring on Com86*

คลาส **Monitoring on Com86** เป็นคลาสย่อยที่เกี่ยวข้องกับคลาส **Monitoring** โดยจะทำงานในส่วนการแสดงผลสถานะการทำงานของอุปกรณ์ที่จะทำงานภายในคอม86 ของโปรแกรมช่วยเหลือผู้ใช้งาน โดยจะส่งข้อมูลไปยังส่วนของคอมพิวเตอร์ โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ mailBoxAddr เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลตำแหน่งเมล์บ็อกที่ใช้ส่งข้อมูลของแต่ละอุปกรณ์
- แอตทริบิวต์ DeviceName เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของอุปกรณ์ชนิดต่างๆที่ผู้ใช้ตั้งให้
- แอตทริบิวต์ MonitorData เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะการทำงานของอุปกรณ์แต่ละตัวที่จะนำไปแสดงผล
- เมธอด SendMonitorData() เป็นเมธอดที่ใช้ส่งค่าข้อมูลสถานะการทำงานของอุปกรณ์แต่ละตัวไปยังคอมพิวเตอร์เพื่อแสดงผลต่อไป
- เมธอด GetMonitorData() เป็นเมธอดที่ใช้รับค่าข้อมูลสถานะการทำงานของอุปกรณ์แต่ละตัวที่ทำงานอยู่ ณ.ปัจจุบัน

4.7 แผนภาพแสดงการทำงานของโปรแกรมจัดการระบบ (Design Flow)

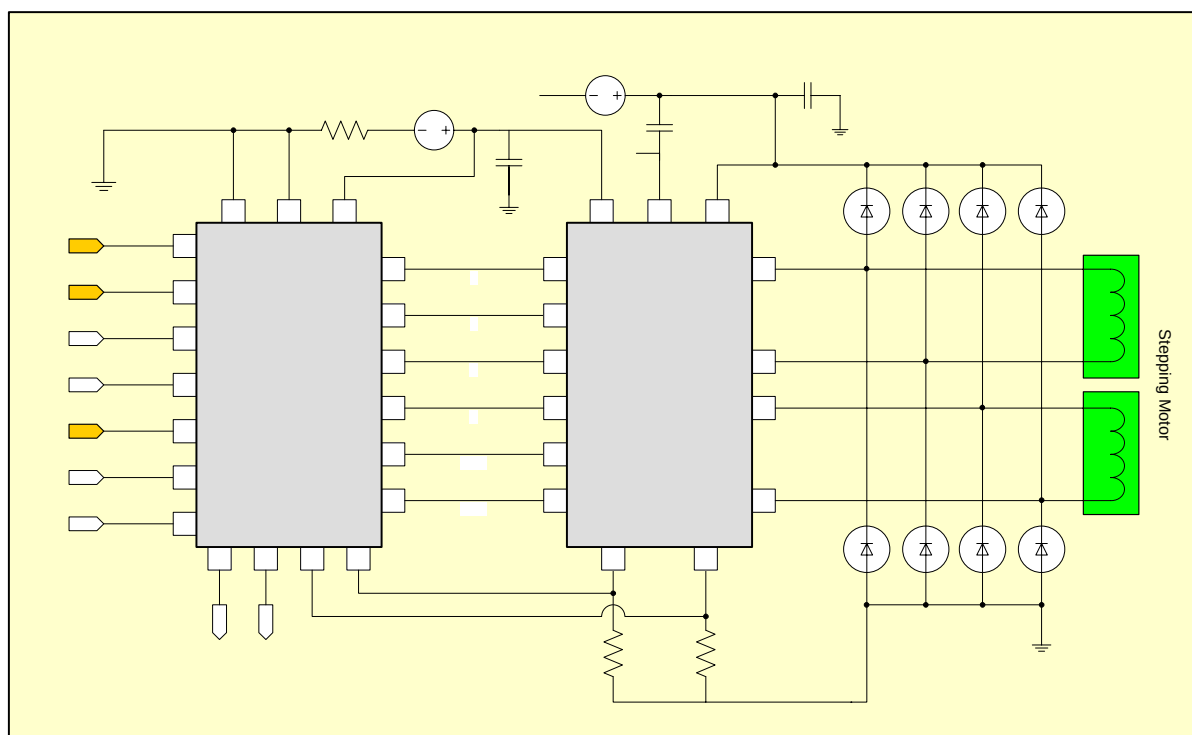


รูปที่ 4-26 แผนภาพแสดงการทำงานของโปรแกรมจัดการระบบ (Design Flow)

4.8 วงจรโมดูลของอุปกรณ์ (Devices Module)

เนื่องจากการเรียกใช้ชุดคำสั่งที่เตรียมไว้นั้น จะต้องมีการกำหนดค่าพอร์ตของอุปกรณ์ ที่จะต้องติดต่อกับไมโครคอนโทรลเลอร์อย่างถูกต้อง และเพื่อให้เข้ากันกับโปรแกรมช่วยเหลือผู้ใช้งานบนคอมพิวเตอร์ ทางคณะผู้จัดทำโครงการจึงได้ทำการออกแบบวงจรสำหรับใช้งานอุปกรณ์ และเป็นวงจรที่สามารถเรียกใช้งานชุดคำสั่งที่เตรียมการไว้อย่างถูกต้อง โดยไม่มีปัญหาแต่อย่างใด

4.8.1 สเตปปีงมอเตอร์

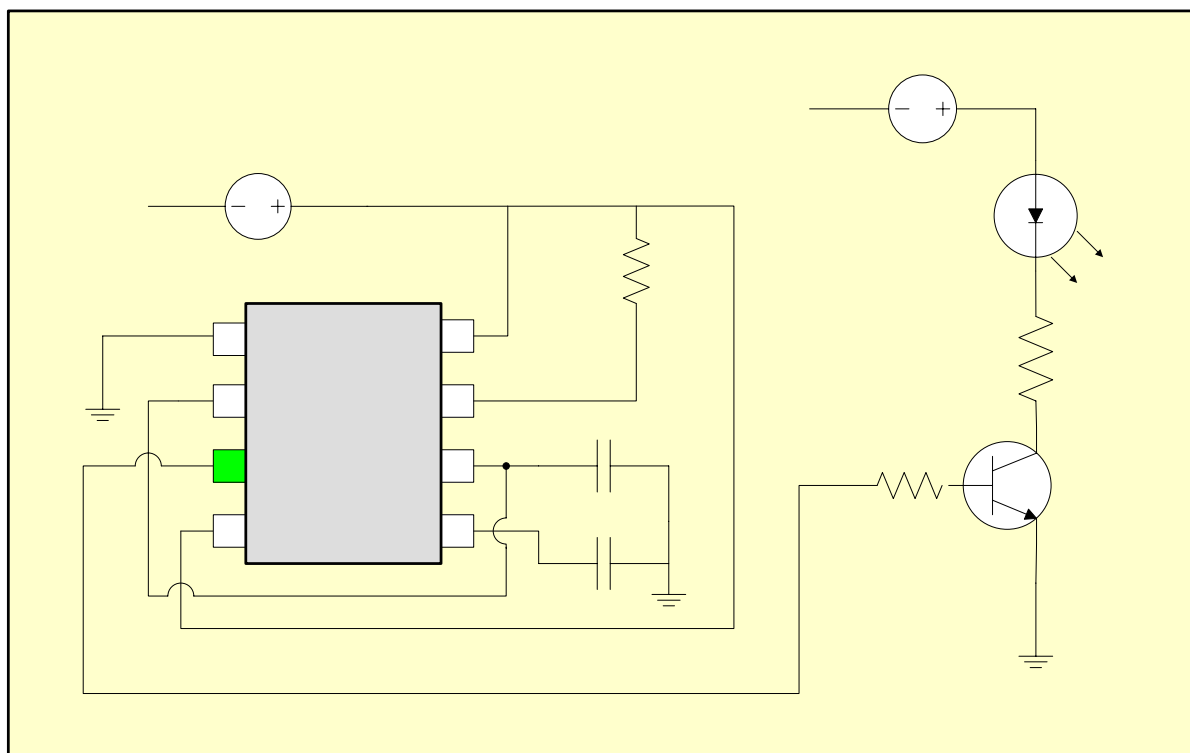


รูปที่ 4-27 แผนภาพแสดงโมดูลของการขับเคลื่อนมอเตอร์ด้วย L297-L298N

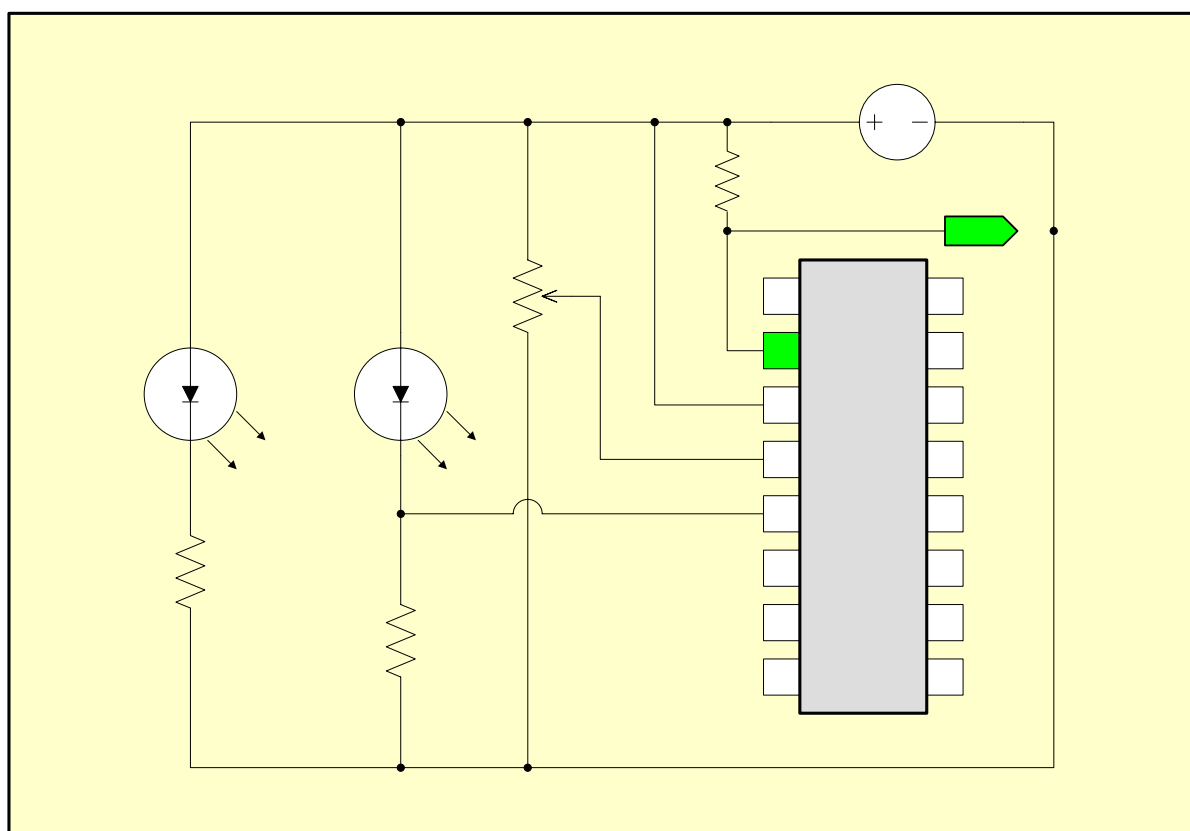
โมดูลทำการขับเคลื่อนอุปกรณ์สเตปมอเตอร์ ตัวนี้ประกอบด้วยไอซี L297 ที่ทำหน้าที่เลื่อนบิตและทำการกำหนดทิศทาง การเลื่อนบิต รวมถึงการเพิ่มบิตที่ใช้งาน ซึ่งเป็นไอซีที่เหมาะสมกับการใช้งานควบคุมสเตปมอเตอร์อย่างมาก โดยขั้นตอนต่อไป ก็คือการเพิ่มกระแสในการขับเคลื่อนมอเตอร์ โดยทางคณะผู้จัดทำได้เลือก L298N ที่เป็นไอซีสำหรับการเพิ่มกระแสในการขับเคลื่อนมอเตอร์ที่รองรับการจ่ายไฟถึง 36 โวลต์ ซึ่งค่อนข้างจะครอบคลุมการใช้งานมอเตอร์เป็นอย่างมาก

โดยในส่วนการเพิ่มกระแส เพื่อขับเคลื่อนมอเตอร์ของไอซี L298N นั้นสามารถประยุกต์ใช้งานกับการขับเคลื่อนมอเตอร์ชนิดอื่น ๆ อีกด้วย โดยการเลือกขั้วอินพุตไปยัง ดีซีมอเตอร์ หรือรวมถึงการสร้างวงจรขับเคลื่อนมอเตอร์แบบ เฮลบริดจ์ (H-Bridge) สำหรับการควบคุมมอเตอร์ดีซี โดยจะไม่ยกตัวอย่างในที่นี้ โดยจะสามารถหาวิธีการจากภาคผนวกท้ายเล่มได้

4.8.2 อินฟราเรดเซนเซอร์



รูปที่ 4-28 แผนภาพแสดงโมดูลของการส่งอินฟราเรดด้วยควมถี่ 38 KHz



รูปที่ 4-29 แผนภาพแสดงโมดูลของวงจรตรวจสอบด้วยอินฟราเรดเซนเซอร์

2

3

LMC